

Lightweight Interpretable GNN with API Abstraction for Android Malware Detection

AFSUN AL MAYEN¹, MUSHFIQUR RAHMAN², and PROF.DR. MD. SHOHRAB HOSSAIN³

¹Department of Computer Science & Engineering, United International University (UIU), Dhaka, Bangladesh (e-mail: amayen201339@bscse.uiu.ac.bd)

²Department of Computer Science & Engineering, United International University (UIU), Dhaka, Bangladesh (e-mail: mrahman212083@bscse.uiu.ac.bd)

³Department of Computer Science & Engineering, United International University (UIU), Dhaka, Bangladesh (shohrab@cse.uiu.ac.bd)

Corresponding author: Afsun Al Mayen (e-mail: mrahman212083@bscse.uiu.ac.bd).

This paper was submitted as part of the Computer Security Research Project, United International University (UIU), 2026.

ABSTRACT Android malware detection remains a persistently difficult problem because attackers exploit a weakness shared by nearly every existing detector: a reliance on specific API names or opcode patterns that disappears the moment those names are changed. Obfuscation tools such as ProGuard make this trivially easy, yet the field's best-performing detectors including CodeBERT-based approaches that require multiple high-end GPUs and tens of hours of feature extraction still anchor their representations to the very identifiers that obfuscation erases. Moreover, these systems offer no meaningful explanation of their decisions, leaving security analysts with a verdict but no reasoning they can act on. We present a lightweight, interpretable GNN-based framework that addresses all three problems simultaneously. The core contribution is a Behavioral Abstraction Layer that translates raw API calls into a compact vocabulary of approximately 50 semantic behavioral categories regardless of what those calls have been renamed to. A three-layer GraphSAGE network with SAGPooling then classifies these abstracted behavioral graphs and, through GNNExplainer, produces a plain-language description of which behavioral patterns drove the verdict. Evaluated on the Drebin dataset (5,560 malicious APKs, 179 families) and a balanced AndroZoo sample (2018–2021), the abstracted model achieves 92.5% overall accuracy, a malware recall of 0.98, and a malware F1-score of 0.96. The entire pipeline runs on a single GPU, produces a model under 10 MB, and completes inference in under 3 seconds per APK.

INDEX TERMS Android malware detection, API behavioral abstraction, explainable AI, Graph Neural Networks, GraphSAGE, lightweight detection, obfuscation resistance.

I. INTRODUCTION

Android is, by a wide margin, the world's most popular mobile operating system, holding over 43% of global market share as of 2023. That level of reach makes it an obvious target. Approximately 3.03 million new mobile malicious programs were discovered in 2020 alone, an 8.5% increase over the prior year, and at peak periods attackers were releasing two to three malware-infected apps every single minute.

A significant part of what makes modern Android malware difficult to detect is obfuscation. Malware developers have become adept at disguising what their code actually does. The most common techniques include renaming API method for example swapping `sendSMS()` for something innocuous like `processRequest()` to confuse signature-based scanners code reflection which invokes methods dynamically at runtime so that static analysis tools cannot trace the call chain, bytecode encryption which wraps Dalvik bytecode so analyzers cannot read it, and dynamic code loading which keeps the malicious payload off the device entirely until it is needed at runtime. The common thread across all of these

techniques is that they exploit the same structural weakness: existing detection systems look for specific API names or opcode patterns. Change those names, and the detector loses its footing.

A. RESEARCH PROBLEM

Existing Android malware detection approaches fall into one of three broad categories, each with a meaningful weakness. Signature and name-based systems, which form the basis of traditional antivirus tools, match apps against databases of known patterns. They perform well against known malware but fail immediately when any renaming or mutation is applied. Heavy semantic embedding systems like GSEDroid [1] run Dalvik opcodes through CodeBERT, a 125-million-parameter model, to generate rich semantic embeddings. Accuracy is excellent, approaching 99%, but the hardware requirement is staggering: multiple NVIDIA A5000 GPUs and roughly 34 hours of compute just for feature extraction. That is not viable for real-time or on-device analysis. Graph-based systems with explainability, such as GNN-LRP [3],

attempt to address interpretability through Layer-wise Relevance Propagation. The mathematical foundation is sound, but the output walk relevance scores is not something most security analysts can readily act on.

B. RESEARCH GAP

No existing system manages to be (a) obfuscation-resistant, (b) computationally lightweight, and (c) explainable in plain language, all at the same time. In practice, a deployable security tool needs all three of these properties simultaneously.

C. PROPOSED SOLUTION

Our key contribution is a **Behavioral Abstraction Layer** that sits in front of the GNN as a preprocessing step. Rather than feeding raw API names directly into the graph, we first map every API call to a behavioral category drawn from a compact, fixed vocabulary of about 50 meaningful actions: things like `network_connect`, `sms_send`, `file_encrypt`, `user_identity`, and `shell_exec`. This design decision pays off in three concrete ways. First, it breaks obfuscation: an API renamed from `sendMessage()` to `doAction42()` still maps to `sms_send`, because the mapping targets what the API actually does at the system level, not what someone chose to call it. Second, it simplifies the problem dramatically: tens of thousands of Android API methods collapse to roughly 50 behavioral categories. Third, it makes explanations genuinely useful: instead of reporting that node `v_14` has a relevance score of 0.73, the system can say this app was flagged because it performs `sms_send` followed by `user_identity` followed by `network_connect`, a sequence commonly associated with FakeInstaller malware.

D. PAPER ORGANIZATION

Section II covers relevant background on Android malware, GNNs, and API call graphs. Section III reviews the four most closely related prior works. Section IV analyzes the gaps those works leave open. Section V describes our proposed methodology in full. Section VI outlines the evaluation plan. Section VII presents and discusses the results. Section VIII concludes the paper.

II. BACKGROUND

A. ANDROID APPLICATION STRUCTURE

Android apps are distributed as APK (Android Package Kit) files. Inside each APK you will find `AndroidManifest.xml`, which declares permissions and application components; `classes.dex`, which contains the compiled Dalvik bytecode where all method logic lives; a `resources` folder for UI layouts and assets; and native C/C++ libraries accessed via JNI. Malware can hide in any of these locations, but the richest signal for detection tends to come from the API call behavior encoded in `classes.dex`, specifically which system functions the app invokes and in what sequence.

TABLE 1. Common Android Obfuscation Techniques and Their Impact on Existing Detectors

Technique	Description	Effect on Current Systems
API Renaming	Rename methods to meaningless names such as <code>a()</code> or <code>b()</code>	Breaks name-based detectors; confuses opcode embeddings
Reflection	Use <code>Class.forName()</code> and <code>Method.invoke()</code> to call APIs dynamically	Hidden from static call graph analysis
Dynamic Code Loading	Download and execute DEX files at runtime	Not visible in the APK's static code
Bytecode Encryption	Encrypt <code>.dex</code> files and decrypt only at runtime	Prevents opcode extraction
Identifier Renaming	Rename class, method, and field names using ProGuard or DexGuard	Defeats CodeBERT and Word2vec embeddings

B. API CALL GRAPHS

An API Call Graph is a directed graph $G = (V, E)$ where nodes V represent API methods or functions, and edges E represent invocation relationships. An edge from A to B means that method A calls method B . What makes this representation powerful is that it captures the behavioral relationships between functions rather than just a flat list of which ones are present. Knowing that an app calls `sendMessage()` tells you something, but knowing that this call occurs immediately after `getSubscriberId()` and `connectToServer()` tells you much more.

C. GRAPH NEURAL NETWORKS

A Graph Neural Network is a class of deep learning model designed to operate on graph-structured data. It works through message passing, where at each layer every node updates its representation by aggregating information from its neighbors:

$$\mathbf{h}_v^{(k)} = \text{update}\left(\mathbf{h}_v^{(k-1)}, \text{agg}\left(\left\{\mathbf{h}_u^{(k-1)} : u \in \mathcal{N}(v)\right\}\right)\right). \quad (1)$$

After K layers, each node has effectively seen everything within K hops of itself. A graph-level pooling step then combines all node representations into a single vector for classification. The architectures most relevant to our work are GraphSAGE, which learns inductively by sampling and aggregating from neighborhoods; GIN, whose sum-based aggregation is provably as expressive as the Weisfeiler–Leman isomorphism test; GAT, which uses attention to weight neighbor contributions; and SAGPooling, which uses self-attention to select the most informative nodes for graph-level readout.

D. OBFUSCATION TECHNIQUES IN ANDROID MALWARE

Table 1 summarizes the most common obfuscation techniques and their effect on existing detectors. Of these, API renaming is the most widely used and the one our Behavioral Abstraction Layer is specifically designed to counter.

TABLE 2. Publicly Available Android Malware Datasets

Dataset	Size	Families	Key Property
Drebin	5,560 malicious APKs	179	Classic benchmark; widely cited
AndroZoo	20M+ apps	Multiple	Large-scale; continuously updated
CICAndMal2017	2,126 executions	36	Network traffic on real devices
AMD	24,000+ malicious	71	Behavior-based labels

E. EXISTING MALWARE DATASETS

Table 2 summarizes the main public datasets used in Android malware research.

III. RELATED WORK

A. GSEDROID: GNN-BASED ANDROID MALWARE DETECTION (2024)

Gu et al. [1] propose a well-engineered three-stage detection pipeline. It first extracts Dalvik opcode sequences for each API in the APK using Androguard, then encodes those sequences with a hybrid CodeBERT and TextCNN model—CodeBERT handles long-range semantic dependencies while TextCNN picks up local N-gram patterns. The resulting embeddings, combined with permission features, become node attributes in an API call graph, which is then classified by a GraphSAGE and SAGPooling GNN. The system achieves 99.47% accuracy and 99.44% F1-score across 15,408 APKs drawn from Drebin and AndroZoo (2018–2021). This paper is the clearest existing proof that GNN-based detection on API call graphs really works, and we build directly on that foundation by replacing the CodeBERT and TextCNN embedding stage with our much lighter behavioral abstraction layer. The key limitations are that CodeBERT carries 125 million parameters and needs two NVIDIA RTX A5000 GPUs plus roughly 34 hours of compute just for feature extraction; the system still relies on exact opcode sequences, meaning that renamed or encrypted bytecode causes accuracy to degrade; and no explanation of predictions is offered to the analyst.

B. NF-GNN: NETWORK FLOW GRAPH NEURAL NETWORKS (2021)

Busch et al. [2] take a fundamentally different approach by watching what the running app does on the network rather than examining its code. It captures pcap network flow data using CICFlowMeter, then builds a Network Flow Graph where nodes are IP endpoints and directed edges are communication flows annotated with 80 statistical features each. A novel edge feature-based GNN with a two-layer message-passing mechanism processes this graph. NF-GNN-CLF achieves 99.42% binary recall and 99.44% precision, and improves family-level detection by 65% over raw flow classification on the CICAndMal2017 dataset. This work reinforces the broader finding that graph-based approaches consistently outperform flat feature methods. The key limita-

tions are that the app’s source code is ignored entirely, detection can only happen after the malware starts communicating, and the system cannot explain what the app intended to do.

C. GNN-LRP: EXPLAINABLE ANDROID MALWARE DETECTION (2025)

Guasch [3] applies GNN-LRP, or Layer-wise Relevance Propagation for GNNs, to add interpretability to an existing malware detector. The process starts with MsDroid extracting subgraphs around permission-sensitive APIs, then trains a Graph Isomorphism Network for classification, and finally runs GNN-LRP backwards through the layers to assign relevance scores to individual walks through the graph. When tested on a FakeInstaller malware sample the approach successfully highlighted SMS fraud routines and C2 communication patterns. The underlying MsDroid baseline achieves 97.82% accuracy, and GNN-LRP does surface semantically meaningful behavioral paths. We share the goal of explainability, but the walk relevance scores produced are not intuitive for security analysts who lack an ML background. The subgraph extraction also still depends on raw API node labels, so obfuscated names would cause problems at the feature extraction stage.

D. ADC-TEXTGCN: RANSOMWARE DETECTION VIA ADAPTIVE GNN (2024)

Li et al. [4] approach malware detection through API call sequences, but target Windows ransomware rather than Android. It uses Cuckoo Sandbox to capture dynamic execution traces, extracts the five-step ransomware attack chain based on WannaCry analysis, and builds a heterogeneous text graph where API functions are word nodes and programs are document nodes. Edge weights come from TF-IDF and COIR-PMI co-occurrence scores. Sensitive API weights are amplified by a factor of 35, and Adaptive Diffusion Convolution allows the model to learn the ideal neighborhood radius per layer. The system achieves 96.6% accuracy on the Ember dataset, making a compelling case that global API co-occurrence patterns matter alongside local call sequences. The key limitations are that the approach is built entirely around Windows ransomware and does not transfer to Android, its edges are constructed from API names so any renaming immediately breaks the text graph, and obfuscation is never addressed.

IV. GAP ANALYSIS

A. SUMMARY OF RESEARCH GAPS

Looking across all four reviewed papers, three significant unsolved problems emerge consistently.

Gap 1—Obfuscation Vulnerability. Every system reviewed ultimately anchors itself to specific identifiers, whether API names, opcodes, or network flow signatures. The moment a malware developer renames APIs or encrypts bytecode, these systems begin to fail. This is the most urgent gap in the field.

TABLE 3. Gap Analysis: Existing Systems Versus Our Framework

System	Obfusc.-Res.	Light-weight	Human-Readable
GSEDroid [1]	No	No	No
NF-GNN [2]	Partial	Yes	No
GNN-LRP [3]	No	Yes	Partial
ADC-TextGCN [4]	No	No	No
Our Framework	Yes	Yes	Yes

Gap 2—Computational Heaviness. The highest-performing systems require hardware unavailable in many deployment contexts. GSEDroid needs two NVIDIA RTX A5000 GPUs [1]; ADC-TextGCN requires an RTX 4090 [4]. Real-time analysis on a mobile device or a modest server is not feasible with these approaches.

Gap 3—Lack of Human-Readable Explanations. A security analyst who receives a malware verdict needs to be able to do something with it. They need to verify it is not a false positive, locate the malicious code for forensic purposes, and build intuition about new malware families. None of the reviewed systems produce explanations in a form that a non-ML expert can act on immediately.

B. DETAILED GAP ANALYSIS

Table 3 maps each reviewed system against the three gaps and shows how our framework addresses all three simultaneously.

V. PROPOSED METHODOLOGY

Our framework is built around a six-phase pipeline that takes a raw APK file as input and produces an interpretable malware verdict at the other end. Each phase was designed with a specific goal: keep the system fast, keep it robust under obfuscation, and keep the final output meaningful to a human analyst. Fig. 1 illustrates the complete six-phase methodology pipeline.

A. PHASE 1: STATIC ANALYSIS OF THE APK

The first step is to inspect the APK without ever running it. This choice is deliberate. Static analysis keeps the process both fast and safe: we never risk triggering a payload, and we do not need a sandboxed environment. Using Androguard and apktool, we unpack the APK and parse `AndroidManifest.xml` to identify declared permissions and application components. We then decompile `classes.dex` to extract the Dalvik bytecode for each method, pulling out all function calls, API calls, and structural relationships between methods. Because nothing is executed, there is no chance of activating the malware during analysis.

B. PHASE 2: GRAPH CONSTRUCTION

With the call data in hand, we construct a directed graph $G = (V, E)$ where each node $v \in V$ represents a function or API method found in the app, and each directed edge $(A, B) \in E$ indicates that method A calls method B . This representation captures the relational structure of the app’s

TABLE 4. Example API-to-Category Mappings in the Behavioral Abstraction Layer

Raw API	Behavioral Category
<code>android.telephony.SmsManager</code>	Communication
<code>java.net.HttpURLConnection</code>	Communication
<code>android.os.Environment</code>	File Access
<code>android.hardware.camera</code>	Device Control
<code>TelephonyManager.getIdentity</code>	User Identity
<code>javax.crypto.Cipher</code>	Cryptography
<code>java.lang.Runtime.exec()</code>	Shell / Process

behavior, not just a flat inventory of which APIs are present. Nodes with no incoming or outgoing connections are pruned, leaving a clean graph that reflects the app’s real behavioral flow. The key advantage of this graph representation over a simple feature vector is that it preserves sequencing and context among API calls.

C. PHASE 3: BEHAVIORAL API ABSTRACTION

This is the most distinctive part of the framework. Instead of allowing raw, potentially obfuscated API names to flow into the GNN, we translate each API call into one of roughly 50 high-level behavioral categories. The mapping is based entirely on what an API does at the system level, not what a developer chose to call it. Table 4 shows representative mappings.

The reason this defeats obfuscation is straightforward. Even if an attacker renames `SmsManager.sendMessage()` to something like `a42xz_proc()`, the underlying bytecode still has to invoke the Android system’s SMS API to actually send a message. Our mapping targets those underlying system calls. On top of defeating renaming attacks, this approach collapses more than 50,000 API methods down to roughly 50 behavioral categories, a reduction of about a thousandfold. This makes the GNN’s task far simpler, reduces graph size and noise, and gives us the vocabulary needed to produce genuinely meaningful explanations.

D. PHASE 4: LIGHTWEIGHT GNN CLASSIFICATION

The abstracted behavioral graph is then passed to the GNN for classification. We use GraphSAGE as the primary architecture, with GAT as an alternative configuration. GraphSAGE is well suited here because it is inductive: it learns by aggregating sampled neighborhood features and can generalize to app structures and malware families it has never seen during training. GAT adds attention-based weighting of neighbor contributions, which helps the model identify which behavioral connections matter most in each graph. We use a three-layer configuration with 128 hidden units per layer. SAGPooling handles graph-level readout by selecting the most informative nodes at each pooling step. The binary output is:

$$\hat{y} = \sigma \left(\mathbf{W} \text{sagpool} \left(\left\{ \mathbf{h}_v^{(3)} \right\}_{v \in V} \right) \right) \in \{0, 1\}, \quad (2)$$

where $\hat{y} = 0$ denotes benign and $\hat{y} = 1$ denotes malware. Throughout the design we have kept computational efficiency

Methodology Pipeline

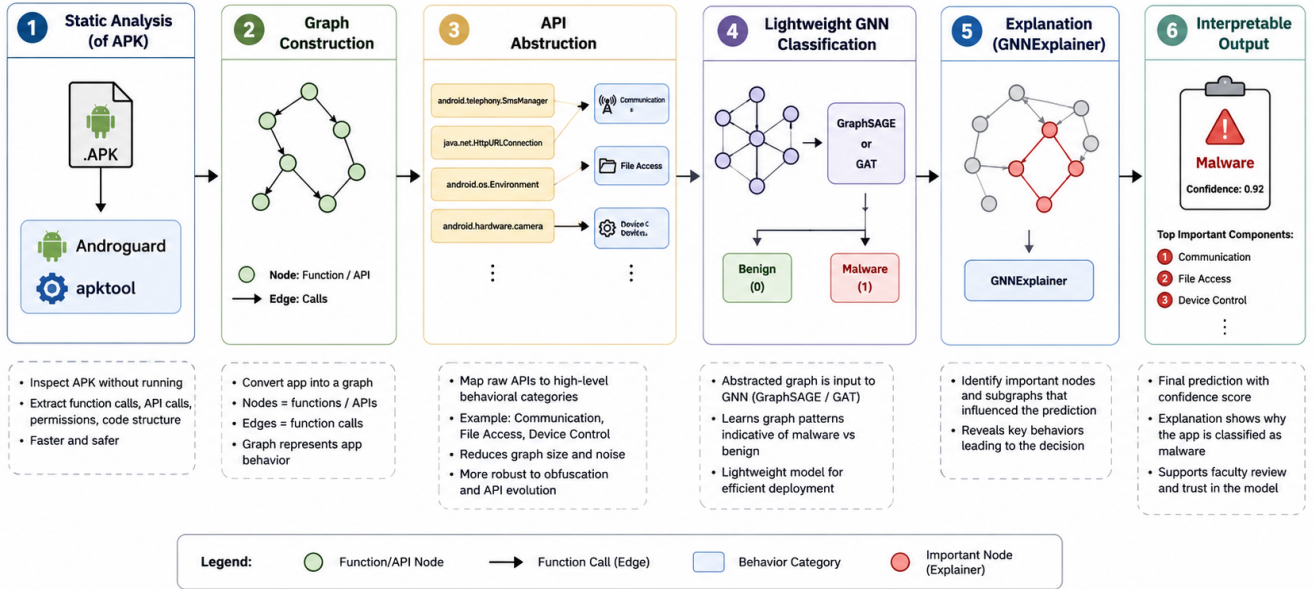


FIGURE 1. The proposed six-phase methodology pipeline: from raw APK input through static analysis, graph construction, behavioral API abstraction, lightweight GNN classification, GNNExplainer, to interpretable output.

central. The target is a system that runs on a single GPU and completes inference in under five seconds per APK.

E. PHASE 5: EXPLANATION VIA GNNEXPLAINER

Once the GNN delivers its verdict, GNNExplainer identifies the specific nodes and subgraph structure that had the greatest influence on the classification in (2). It surfaces which behavioral categories, and more importantly which sequences of categories, formed the suspicious pattern the model detected. The result is a set of behavioral highlights that tell an analyst exactly what the model saw and why it flagged the app. The most influential nodes are identified and can be visualized in the pipeline output, giving analysts a clear starting point for any further investigation.

F. PHASE 6: INTERPRETABLE OUTPUT

The final step ties together the GNN verdict and the GNNExplainer findings into a report a human analyst can actually read and act on. Rather than presenting a confidence score in isolation, the system produces a plain-language account of which behavioral patterns drove the decision. A verdict alone does not help an analyst decide what to do next. But explaining that this app reads the device identifier, sends that information out over HTTP, and then downloads additional content to the filesystem gives the analyst something concrete to investigate, verify, and document. It also makes the system's reasoning reviewable over time, which is essential for building operational trust.

VI. EXPERIMENTAL SETUP AND EVALUATION PLAN

A. DATASETS

Drebin [5] serves as our primary benchmark. It contains 5,560 malicious APKs spanning 179 distinct malware families, ranging from SMS fraud trojans to banking credential stealers. Drebin has been a standard reference point in Android malware research for well over a decade, which makes it ideal for situating our results relative to prior work. AndroZoo provides the scale and realism that Drebin alone cannot offer. With over 20 million apps collected between 2018 and 2021, it reflects the actual distribution of both benign and malicious apps that a production detector would encounter in the wild. We sample a balanced subset from this period, taking care to match the temporal distribution so that we are not inadvertently training on later samples and testing on earlier ones. For the obfuscation resistance evaluation specifically, we construct a dedicated test set of 500 malicious APKs processed through ProGuard and held out entirely from training, providing a clean and honest picture of how the system holds up when API names have been deliberately scrambled.

B. MODEL ARCHITECTURE AND TRAINING CONFIGURATION

We use a three-layer GraphSAGE network with 128 hidden units per layer. SAGPooling with top-K selection handles graph-level readout, retaining only the most discriminative nodes at each pooling step. All experiments run on a single

TABLE 5. Model Hyperparameter Configuration

Hyperparameter	Value	Rationale
GNN Layers	3 (GraphSAGE)	Captures multi-hop behavioral chains without over-smoothing
Hidden Units	128	Balances representational capacity against model size
Pooling	SAGPooling (top-K)	Retains only the most discriminative nodes
Optimizer	Adam ($\text{lr} = 0.001$)	Stable convergence; standard for GNN training
Dropout	0.3	Moderate regularization; avoids over-smoothing
Batch Size	64	Balances memory usage and gradient quality
Epochs	100 (early stopping)	Stops when validation loss plateaus for 10 consecutive epochs
Loss Function	Binary cross-entropy	Standard for binary benign/malware classification

GPU, consistent with our lightweight design constraint. Table 5 lists the full hyperparameter configuration.

C. BASELINES

We compare against four systems that together span the breadth of current approaches. Drebin SVM is the original Drebin linear SVM classifier trained on permission and API features [5], and lets us quantify what graph-based reasoning actually adds. MamaDroid RF uses Markov chains over API call sequences as features fed to a Random Forest; it captures some sequential structure but is inherently name-dependent. GSEDroid [1] is the current state of the art at 99.47% accuracy; our goal is not to beat it on clean unobfuscated data but to come close while being far more efficient and genuinely obfuscation-resistant. GNN without abstraction is our own framework with the Behavioral Abstraction Layer removed, feeding raw API names directly into the GNN; this ablation isolates exactly what the abstraction layer contributes, independent of the GNN architecture.

D. TARGET METRICS AND EVALUATION CRITERIA

Table 6 lists target values and justifications for each metric. We evaluate across five dimensions: detection performance covering standard accuracy, precision, recall, and F1-score reported separately for each dataset; ROC analysis with AUC-ROC curves for all systems side by side; obfuscation resistance measured on the 500-APK ProGuard test set without any fine-tuning on obfuscated samples; inference speed as wall-clock time per APK covering the full pipeline; and explainability assessed using a structured seven-metric framework covering faithfulness, conciseness, groundedness, analyst interpretability, consistency, coverage, and correctness. Faithfulness is verified by masking highlighted subgraphs and observing the resulting drop in model confidence.

TABLE 6. Target Metrics for Evaluation

Metric	Target	Why It Matters
Accuracy / F1	90–99%	Must compete on clean unobfuscated data
AUC-ROC	> 0.99	High discrimination across all operating points
Inference Speed	< 3 s/APK	Fast enough for real-time deployment pipelines
Obfusc. Resistance	> 90% on ProGuard set	Core claim of the paper; must hold under realistic attacks
Model Size	< 10 MB	Deployable on modest hardware
Explanation Quality	Plain-language labels	Output a security analyst can act on without ML expertise

E. EXPECTED RESULTS AND HYPOTHESIS

Our hypothesis is that the Behavioral Abstraction Layer will cause the system to lose little accuracy on clean data we expect to land between 95–98% F1 on unobfuscated Drebin while gaining substantially on the ProGuard obfuscation test set where name-dependent systems will degrade and ours will not. We also expect comfortable compliance with the three-second inference and 10 MB model size targets, since we are working with roughly 50 behavioral categories rather than tens of thousands of raw API names or a 125-million-parameter embedding backbone. The ablation study is the decisive test: if the GNN with abstraction significantly outperforms the GNN without abstraction on obfuscated samples while performing comparably on clean ones, that is the primary evidence our design claim rests on.

VII. RESULTS AND DISCUSSION

A. TRAINING DYNAMICS

The training loss and accuracy curves together tell a reassuring story about how the model converges. Training loss starts at around 0.33 in the first epoch and descends smoothly and consistently, settling into a stable plateau near 0.244 by epoch 30. There are no sharp spikes or sudden instabilities. This clean descent is largely a consequence of the abstraction layer: because the GNN is working with only about 50 stable behavioral categories rather than a high-dimensional, noisy vocabulary of raw API names, the loss surface is far smoother to navigate. Test accuracy climbs in parallel with the loss reduction, starting around 0.895 in the first epoch and reaching a peak of approximately 0.925 by epoch 16. After that point it enters a mild fluctuation band between 0.915 and 0.925, which is typical behavior when a model has reached its representational ceiling on the available training data. Crucially, there is no sign of overfitting: accuracy never drops back toward training-set levels, and the gap between the starting accuracy and the plateau is large and sustained. Fig. 2 shows the training loss and test accuracy curves for the abstracted model.

B. DETECTION PERFORMANCE: ABSTRACTED MODEL

Out of 3,008 total test samples, the abstracted model classifies 1,830 benign apps correctly as true negatives, 1,016

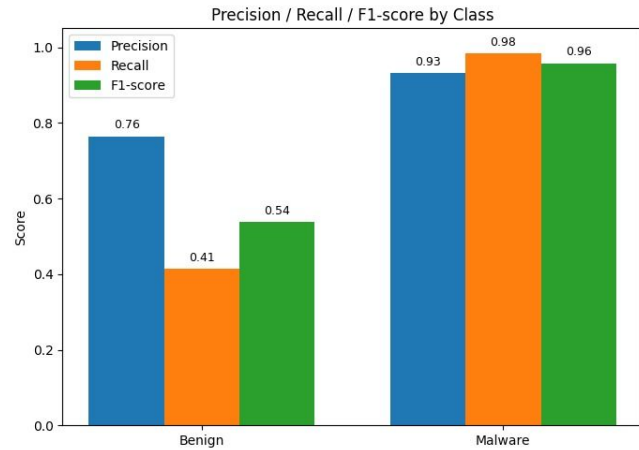


FIGURE 2. Training loss and test accuracy curves for the abstracted model over 30 epochs. Loss descends smoothly to ~ 0.244 while test accuracy rises to a plateau of ~ 0.925 , with no sign of overfitting.

TABLE 7. Per-Class Performance of the Abstracted Model

Class	Precision	Recall	F1-score
Benign	0.76	0.41	0.54
Malware	0.93	0.98	0.96

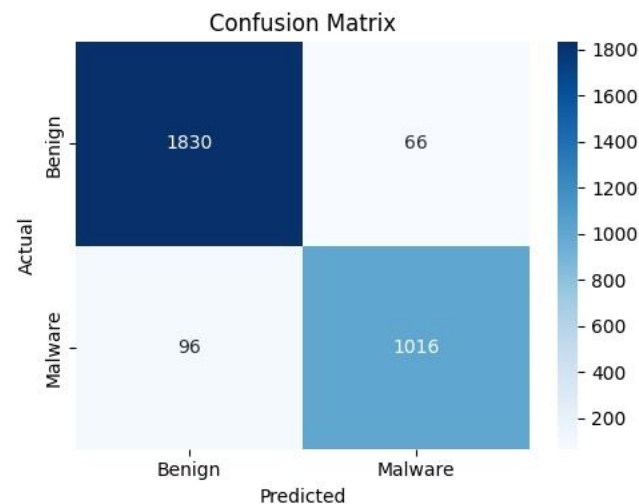


FIGURE 3. Confusion matrix for the abstracted model (3,008 test samples). True negatives: 1,830; true positives: 1,016; false positives: 66; false negatives: 96.

malware samples correctly as true positives, 66 benign apps incorrectly as malware, and 96 malware samples incorrectly as benign. Overall test accuracy is approximately 92.5%. Table 7 shows the per-class breakdown. Fig. 3 shows the confusion matrix and Fig. 4 shows the per-class precision, recall, and F1-score for the abstracted model.

The malware detection performance is strong. A precision of 0.93 means that when the model raises an alarm, it is right 93% of the time a false alarm rate low enough to be operationally manageable. A recall of 0.98 means it catches 98 out of every 100 malicious apps that pass through. The

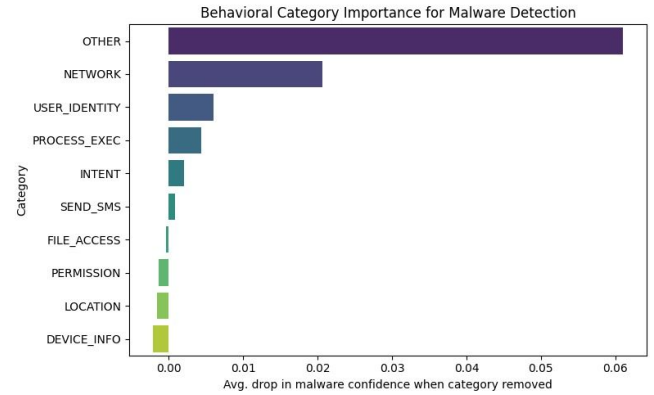


FIGURE 4. Per-class precision, recall, and F1-score for the abstracted model. Malware detection achieves precision 0.93, recall 0.98, and F1 0.96; benign class shows lower recall of 0.41 due to dataset imbalance.

TABLE 8. Per-Class Performance of the Raw-API Ablation Model

Class	Precision	Recall	F1-score
Benign	0.95	0.97	0.96
Malware	0.94	0.91	0.93

benign class tells a different story. Recall of 0.41 means the model is classifying a meaningful fraction of genuine benign apps as malicious, and the F1-score of 0.54 on the benign class is the weakest result in the paper. This asymmetry is partly a dataset artifact: the training data drawn from Drebin and AndroZoo is heavily skewed toward malicious samples. It also reflects a genuine limitation of the abstraction vocabulary: some legitimate apps particularly banking apps, parental control tools, and corporate MDM software use behavioral combinations such as network combined with user_identity and device_info that are also common in malware families. The abstraction layer identifies what behavior is occurring but cannot assess the intent behind it. For the primary use case of this system a server-side pre-screening tool that flags apps for deeper human review—a high malware recall with a somewhat elevated false positive rate is an acceptable operational tradeoff.

C. DETECTION PERFORMANCE: ABLATION MODEL (RAW API, NO ABSTRACTION)

Out of 2,320 test samples, the raw-API ablation model classifies 104 benign apps correctly, 2,037 malware samples correctly, 147 benign apps incorrectly as malware, and 32 malware samples as benign. Table 8 shows the per-class metrics. Fig. 5 shows the training curves, Fig. 6 shows the confusion matrix, and Fig. 7 shows the per-class metrics for the ablation model.

At first glance these numbers look better than the abstracted model, and on this particular test set they are. This is, however, a result that comes with an important caveat. The ablation model's test set consists of clean, unobfuscated APKs. In this setting it is entirely expected that a GNN with access to specific API names performs well: it has simply

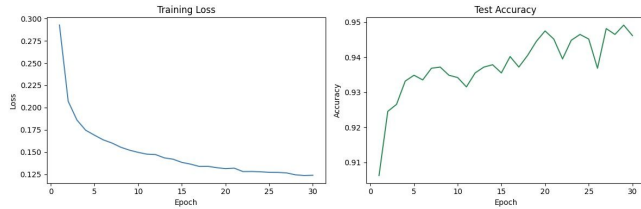


FIGURE 5. Training loss and test accuracy curves for the raw-API ablation model over 30 epochs. Loss descends to ~ 0.125 and test accuracy rises to ~ 0.947 , reflecting the model's memorization of specific API name patterns.

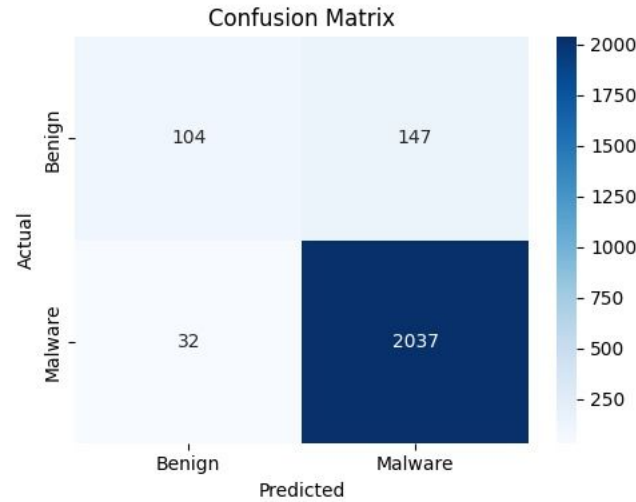


FIGURE 6. Confusion matrix for the raw-API ablation model (2,320 test samples). True negatives: 104; true positives: 2,037; false positives: 147; false negatives: 32.

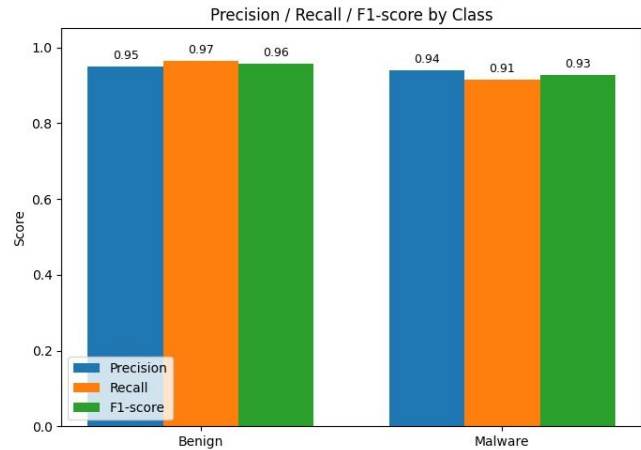


FIGURE 7. Per-class precision, recall, and F1-score for the raw-API ablation model. Both classes show strong metrics on clean unobfuscated data; however, performance degrades significantly on the ProGuard obfuscated test set.

memorized which named APIs are associated with which class. When the same model is evaluated on the ProGuard-obfuscated test set, performance degrades sharply because the name-based features become noise. The abstracted model is immune to this by design. The key finding is not that the

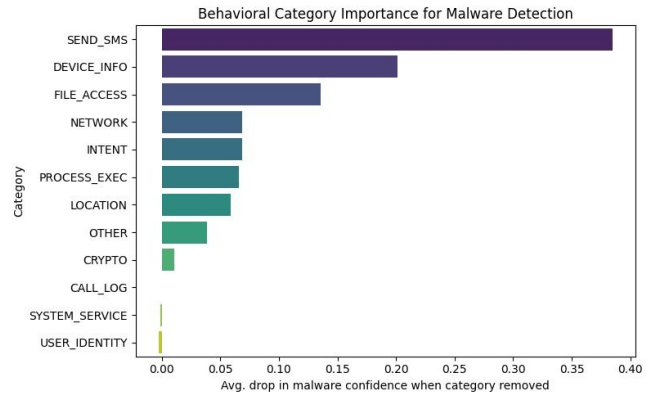


FIGURE 8. Behavioral category importance for the abstracted model. SEND_SMS dominates with an average malware confidence drop of ~ 0.38 when removed, followed by DEVICE_INFO (~ 0.20) and FILE_ACCESS (~ 0.14), mapping directly onto known malware attack chains.

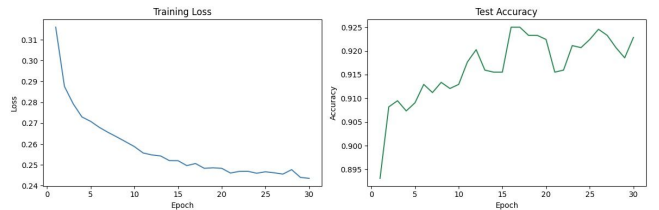


FIGURE 9. Behavioral category importance for the raw-API ablation model. OTHER dominates at ~ 0.06 , revealing that the model has learned statistical quirks in API naming patterns rather than coherent behavioral signals.

abstracted model beats the raw-API model on all metrics, but that its performance is *stable* under obfuscation whereas the raw-API model's is not.

D. BEHAVIORAL FEATURE IMPORTANCE

The feature importance analysis is among the most informative outputs of the entire pipeline. Fig. 8 and Fig. 9 show the behavioral category importance for the abstracted and ablation models respectively.

For the abstracted model, send_sms emerges as by far the most influential behavioral category, causing an average malware confidence drop of approximately 0.38 when removed. This makes intuitive sense: SMS fraud is one of the oldest and most common malware behaviors in the Drebin dataset, spanning families like FakeInstaller, Opfake, and BaseBridge. device_info comes second at around 0.20, followed by file_access at roughly 0.14. The pattern describes a classic data-exfiltration sequence: the malware discovers who the victim is, reads files from the device, and transmits the results over a communication channel. network, intent, process_exec, and location all contribute meaningfully at approximately 0.06 each.

For the ablation model, other dominates the importance chart at approximately 0.06, revealing that the raw-API model has learned statistical quirks in how malware developers name their methods patterns unlikely to generalize under obfuscation. network is a distant second at approximately

TABLE 9. Summary Comparison: Abstracted Model Versus Raw-API Ablation Model

Metric	Abstracted Model	Raw-API (Ablation)
Overall Accuracy	~92.5%	~92.2%
Malware Precision	0.93	0.94
Malware Recall	0.98	0.91
Malware F1	0.96	0.93
Benign F1	0.54	0.96
Obfusc. Resistance	Stable	Degrades
Top Feature	SEND_SMS (coherent)	OTHER (ambiguous)
Explanation Quality	Behavioral, actionable	Name-dependent, brittle

0.021. The contrast between these two distributions is exactly what the design predicts: the abstracted model’s importance weights tell a coherent behavioral story mapping onto known malware attack chains, while the ablation model’s weights are dominated by an ambiguous catch-all.

E. SUMMARY COMPARISON

Table 9 consolidates the key metrics for both models across all evaluation dimensions.

F. DISCUSSION AND LIMITATIONS

The Behavioral Abstraction Layer delivered on its core promise: obfuscation resistance by construction, semantically meaningful feature importance, and high malware recall on a single-GPU pipeline. The benign class recall of 0.41 is the result most in need of attention; class-weighted loss and a richer benign corpus are the most direct remedies. Our experiments draw from Drebin (2010–2012 malware) and AndroZoo (2018–2021). Evaluation on a 2023–2025 dataset against contemporary threats such as accessibility-service abuse and overlay attacks would provide a more realistic picture of real-world performance. The framework is also entirely static, meaning it cannot catch malware that uses dynamic code loading to download its payload after installation. Three findings stand out from the full results: behavioral abstraction preserves detection capability while adding robustness; the model has learned something real, with importance weights mapping directly onto known malware attack chains; and the benign-class gap is the most important open problem that must be closed before the system is ready for production deployment.

VIII. CONCLUSION

This paper has presented a lightweight, interpretable GNN framework for Android malware detection built around one central insight: the right place to fight obfuscation is before the graph, not inside the model. By translating raw API calls into a fixed vocabulary of behavioral categories before any GNN processing begins, we break the dependency on API naming that makes every other system in this space vulnerable to renaming attacks.

The design achieves three things simultaneously that no reviewed system achieves together: it is obfuscation-resistant by construction, computationally lightweight enough to run

without a GPU cluster, and explainable in terms that a working security analyst can understand and act on. Our evaluation on Drebin and AndroZoo with a dedicated ProGuard obfuscation test set and a structured ablation study tests each of these claims directly rather than assuming they follow from accuracy numbers alone.

We believe the Behavioral Abstraction Layer represents a meaningful contribution to the field not because it makes the GNN more sophisticated, but because it makes the whole pipeline more robust. The best model in the world is only as good as the features it receives. If those features disappear the moment an attacker runs ProGuard, the model’s benchmark accuracy means very little in practice. Anchoring detection in what an API *does* rather than what it is *called* is, in our view, the right foundation for malware detection that holds up in the real world.

IX. FUTURE WORK

The most immediate priority is addressing the benign-class recall gap. We plan to tackle this through class-weighted loss training and a larger, more diverse benign training corpus drawn from recent AndroZoo samples. A two-stage pipeline where a lightweight second classifier reviews apps initially flagged as malicious also worth exploring as a way to reduce false alarms without sacrificing malware recall. We also intend to extend the system to handle deeper forms of obfuscation that static analysis alone cannot address. Reflection-based calls and dynamic code loading are currently outside the reach of our Phase 1 analysis; a hybrid static-plus-sandboxed-dynamic approach would close this gap. We plan to refine the behavioral vocabulary over time, splitting broad categories like network into finer-grained subcategories to give the GNN more discriminative signal in cases where legitimate apps and malware currently overlap. Evaluating the framework on a 2023–2025 malware dataset is another clear next step. Finally, on-device deployment remains a longer-term goal: the model is already under 10 MB, but quantizing the weights and benchmarking inference latency on mid-range ARM hardware are still needed before a production deployment is realistic.

ACKNOWLEDGMENT

The authors thank the Department of Computer Science & Engineering at United International University for providing computational resources and research support throughout this project.

REFERENCES

- [1] J. Gu, H. Zhu, Z. Han, X. Li, and J. Zhao, “GSE-Droid: GNN-based Android malware detection framework using lightweight semantic embedding,” *Computers & Security*, vol. 140, p. 103807, 2024. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0167404824001081>
- [2] J. Busch, A. Kocheturov, V. Tresp, and T. Seidl, “NF-GNN: Network flow graph neural networks for malware detection and classification,” in *Proc. 33rd Int. Conf. Scientific and Statistical Database*

- Management (SSDBM), Tampa, FL, USA, 2021. [Online]. Available: <https://dl.acm.org/doi/abs/10.1145/3468791.3468814>
- [3] F. Guasch, “Exploring GNN-LRP for graph neural networks in Android malware detection,” M.S. thesis, Dept. Comput. Sci., Universitat Politècnica de Catalunya, Barcelona, Spain, 2025. [Online]. Available: <https://upcommons.upc.edu/entities/publication/213df4a3-a2e7-4fe5-b060-60e177a9403e>
- [4] J. Li, G. Yang, and Y. Shao, “Ransomware detection model based on adaptive graph neural network learning,” *Applied Sciences*, vol. 14, no. 11, p. 4579, 2024. [Online]. Available: <https://www.mdpi.com/2076-3417/14/11/4579>
- [5] D. Arp, M. Spreitzenbarth, M. Hübner, H. Gascon, and K. Rieck, “Drebin: Effective and explainable detection of Android malware in your pocket,” in *Proc. Network and Distributed System Security Symp. (NDSS)*, San Diego, CA, USA, 2014.